

Samsung Galaxy S21 FE → Debian Homelab Server

Step-by-step build documentation, troubleshooting, and lessons learned

Device: Samsung Galaxy S21 FE 5G — SM-G990B2/DS (Snapdragon 888)

Software: Android 16 / One UI 8.0 · Debian GNU/Linux 13 (Trixie) · ARM64

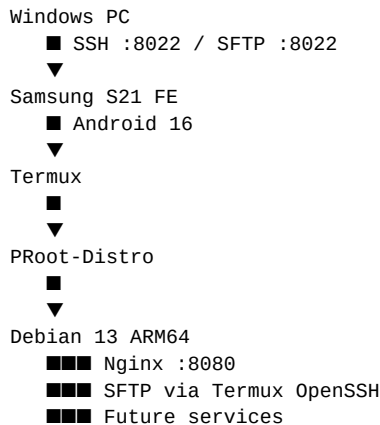
Architecture: Android → Termux → PRoot-Distro → Debian

Project outcome

The phone was converted into a practical Linux homelab server without replacing Android or unlocking the bootloader. Debian runs as an ARM64 Linux userspace inside Termux using PRoot-Distro. The setup provides SSH remote administration, Nginx web hosting, and SFTP access.

Important limitation: this is not bare-metal Linux. Android still owns the kernel, hardware drivers, power management, and network hardware.

1. Final architecture



The S21 FE's USB-C port was not working during the project, so the server remained Wi-Fi based. The phone was connected to another phone's hotspot, so its IP was DHCP-assigned and could change.

2. Verify the device

Device information

Settings → About phone was used to confirm the exact model. The device reported SM-G990B2/DS. Software information showed One UI 8.0, Android 16, and build BP2A.250605.031.A3.G990B2XXSJZF1.

Architecture check

```
uname -m
# Expected:
aarch64

getprop ro.product.model
# Expected:
SM-G990B2

getprop ro.build.version.release
# Expected:
16
```

The aarch64 result confirmed that the phone can run ARM64 Linux userspace software.

3. Why we did not install bare-metal Linux

The initial goal included a proper 24/7 Linux server. However, the phone's OEM unlocking option was completely missing under Developer options on One UI 8.0. Because unlocking/flashing would be destructive and device-specific, the safer route was chosen: retain Samsung's Android kernel and run Debian in userspace.

- No bootloader unlocking was performed.
- No custom kernel or recovery was flashed.
- This greatly reduced the risk of bricking the phone.
- The resulting environment is suitable for many server workloads but does not have unrestricted hardware/kernel access.

4. Install Termux

Termux was installed from a trusted official distribution source. The important rule is to keep Termux and its add-ons from the same signing source.

```
pkg update
pkg upgrade -y
```

```
pkg install proot-distro -y
```

5. Install Debian 13

```
proot-distro install debian
proot-distro login debian

cat /etc/os-release
uname -m
```

The result was Debian GNU/Linux 13 (Trixie), aarch64.

6. Prepare Debian

```
apt update && apt full-upgrade -y
apt install -y sudo nano curl wget git httpd iproute2 net-tools ca-certificates procps
```

Resource checks showed approximately 7.2 GiB RAM visible to Debian, about 6.6 GiB available at the time of testing, six CPU cores visible, and roughly 200 GB available in the reported filesystem. These are more than adequate for lightweight homelab services.

7. SSH server

OpenSSH was installed in Termux rather than trying to make Debian's PRoot environment own the Android network stack.

```
pkg install openssh -y
passwd
sshd
pgrep -a sshd
```

Termux SSH uses port 8022 in this setup. The Termux username discovered during setup was u0_a339. The phone's hotspot address during testing was 10.124.197.28.

```
# From Windows PowerShell
ssh -p 8022 u0_a339@10.124.197.28
```

The first SSH connection succeeded, providing remote administration from Windows.

8. Challenge: Termux service manager

After installing termux-services, sv-enable initially failed because SVDIR was empty. The service directory itself existed, but the service environment had not been initialized in that shell.

```
echo $SVDIR
# Initially empty

source $PREFIX/etc/profile.d/start-services.sh

echo $SVDIR
# /data/data/com.termux/files/usr/var/service
```

After initialization, sshd was enabled with termux-services.

9. Challenge: SSH port already in use

The service-managed sshd initially stayed down because a manually started sshd was already listening on port 8022.

```
sv status sshd
# down: sshd ... want up

# Log showed:
# Bind to port 8022 ... failed: Address already in use
```

Resolution: stop the manually started process and let termux-services own the service.

```
pkill sshd
```

```
pgrep -a sshd

sv up sshd
sv status sshd
pgrep -a sshd
```

The status changed to a running service and Windows SSH continued to work.

10. Surviving Android reboot

Termux:Boot was used so the SSH service could start without manually opening Termux. Termux was also given unrestricted battery/background permission where available, and sleeping/deep-sleep restrictions were avoided.

```
mkdir -p ~/.termux/boot
nano ~/.termux/boot/start-server

# Script:
#!/data/data/com.termux/files/usr/bin/sh
termux-wake-lock
sleep 10
source /data/data/com.termux/files/usr/etc/profile.d/start-services.sh

chmod +x ~/.termux/boot/start-server
```

After reboot, SSH worked without manually opening Termux. This confirmed the boot mechanism was functioning.

11. Install and configure Nginx

Nginx was installed inside Debian.

```
apt update
apt install nginx -y
```

Challenge: privileged port 80

Starting Nginx on port 80 failed with permission denied. This is expected because the Debian PRoot environment does not have normal root privileges over Android's kernel/network stack.

```
nginx
# nginx: [emerg] bind() to 0.0.0.0:80 failed (13: Permission denied)
```

Resolution: use unprivileged port 8080.

```
sed -i 's/listen 80 default_server;/listen 8080 default_server;/' /etc/nginx/sites-available/default
sed -i 's/listen \[::\]:80 default_server;/listen [::]:8080 default_server;/' /etc/nginx/sites-available/default
nginx -t
```

Challenge: port 8080 already in use

When nginx was started again, it reported Address already in use. This was because Nginx was already running successfully on 8080. The correct response was not to start a second instance, but to test or reload the existing one.

```
curl http://127.0.0.1:8080
# Returned: Welcome to nginx!

nginx -s reload
```

A HTTP HEAD request returned HTTP/1.1 200 OK, confirming the web server was healthy.

12. Create the custom homelab page

```
mkdir -p /var/www/homelab
nano /var/www/homelab/index.html
```

The page was configured with a simple S21 FE Homelab dashboard and Nginx's document root was changed from /var/www/html to /var/www/homelab.

```
nginx -t
nginx -s reload
curl http://127.0.0.1:8080
```

The custom page was then reachable from Windows at <http://10.124.197.28:8080> while the phone remained on the same hotspot network.

13. SFTP storage

A dedicated Debian storage tree was created:

```
mkdir -p /srv/storage
mkdir -p /srv/storage/uploads
mkdir -p /srv/storage/backups
echo "S21 FE Homelab Storage" > /srv/storage/test.txt
```

The PRoot-Distro layout on this installation was discovered with:

```
proot-distro list
find $PREFIX/var/lib/proot-distro -maxdepth 3 -type d 2>/dev/null
```

The correct rootfs path was found to be:

```
$PREFIX/var/lib/proot-distro/containers/debian/rootfs
```

A Termux-side symlink was created for convenient SFTP access:

```
mkdir -p ~/homelab
ln -s $PREFIX/var/lib/proot-distro/containers/debian/rootfs/srv/storage ~/homelab/storage
ls -la ~/homelab/storage
```

An initial relative SFTP path was incorrect because SFTP resolved it in a different filesystem context. Using the absolute Termux path worked:

```
sftp -P 8022 u0_a339@10.124.197.28
sftp> cd /data/data/com.termux/files/home/homelab/storage
sftp> ls
backups  test.txt  uploads
```

14. Wi-Fi / Airgorah investigation

Airgorah was considered for authorized Wi-Fi security testing. Debian could see an interface named wlan0:

```
ip link
# wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP>
```

However, Wi-Fi control through Linux generic netlink was not available:

```
iw dev
# Failed to connect to generic netlink.

iw list
# Failed to connect to generic netlink.
```

Therefore the internal Android Wi-Fi adapter could not be treated as a normal Linux monitor-mode adapter from this PRoot environment. A USB Wi-Fi adapter with appropriate driver support would normally be the more practical route, but the phone's USB-C port was not working during this project, so that path was deferred.

Do not run monitor-mode or packet-injection commands against networks without explicit authorization.

15. Major challenges and resolutions

Challenge	Resolution
OEM unlocking missing	Avoided bootloader modification; used Termux + PRoot-Distro.
Termux service SVDIR empty	Sourced start-services.sh to initialize the service environment.
sshd service would not start	Found an existing manual sshd using port 8022; stopped it and enabled service-managed sshd.
Android background limits	Used Termux:Boot and termux-wake-lock; adjusted battery/background settings.
Nginx port 80 permission denied	Moved Nginx to unprivileged port 8080.
Nginx 8080 already in use	Recognized Nginx was already running; tested/reloaded instead of starting a second instance.

Challenge	Resolution
Incorrect PRoot rootfs path	Located actual containers/debian/rootfs path using find.
SFTP relative path failed	Used the absolute Termux storage path.
Wi-Fi generic netlink unavailable	Confirmed hardware/kernel limitation; deferred monitor-mode work.
USB-C not working	Kept the server Wi-Fi-only and postponed Ethernet/SSD/USB Wi-Fi.

16. Current working state

- Debian GNU/Linux 13 (Trixie), ARM64 — working.
- SSH via Termux on port 8022 — working and survives reboot.
- Nginx on port 8080 — working.
- Custom homelab web page — working.
- SFTP to the controlled storage directory — working.
- Phone is currently Wi-Fi/hotspot based.
- USB-C Ethernet, USB SSD, and USB Wi-Fi are deferred because the USB-C port was not working.
- Bare-metal Linux was not installed; Android remains the host OS/kernel.

17. Recommended next steps

- Move the phone from a phone hotspot to a normal home router when possible.
- Reserve a DHCP address for the S21 FE on the router rather than forcing a static address inside Debian.
- Repair the USB-C port if practical.
- Add USB-C Ethernet for a more stable wired server connection.
- Add a USB SSD for persistent server data and backups.
- Make Debian-hosted services start automatically after Android boot.
- Add Python/Node.js application hosting and a database if needed.
- Use a VPN such as Tailscale/WireGuard for remote access rather than exposing SSH directly to the public internet.
- Monitor temperature, battery health, storage usage, and service uptime for 24/7 operation.

18. Useful command reference

```
# Start Debian
proot-distro login debian

# Check Debian
cat /etc/os-release
uname -m

# Check Nginx
pgrep -a nginx
curl -I http://127.0.0.1:8080
nginx -t
nginx -s reload

# Check SSH service (Termux)
sv status sshd
sv up sshd

# Keep device awake
termux-wake-lock
```

```
# Windows SSH
ssh -p 8022 u0_a339@PHONE_IP

# Windows SFTP
sftp -P 8022 u0_a339@PHONE_IP
```

19. Final notes

This project successfully turned the Samsung S21 FE into a useful ARM64 Linux homelab server while preserving the Android installation. The most important lesson was to treat Android as the hardware/kernel layer and Debian/Proot as the application/server layer. This provides a surprisingly capable lab environment, but it is not equivalent to a bare-metal Linux server.

For long-term 24/7 use, wired networking, external storage, thermal management, battery/power considerations, and reliable service startup should be addressed before hosting important data or services.